

Contents

guide Toolformer: Language Models Can Teach Themselves to Use Tools	1
0. 先给结论	2
1. 为什么需要工具	2
2. 论文的 desiderata	3
3. API call 的线性化	3
4. 三步数据生成流程	3
5. Step 1: 采样候选 API call	4
6. Step 2: 执行工具	5
7. Step 3: 用 loss improvement 筛选	5
8. 未来 token 权重	5
9. Finetuning 和推理	6
10. 五个工具	6
11. 数据量和阈值	7
12. 实验设置	8
13. LAMA: factual completion	8
14. 数学任务	8
15. 问答任务	9
16. 多语言问答	9
17. 时间任务	9
18. 语言建模能力是否退化	10
19. Scaling: 多大模型才会用工具	10
20. Decoding strategy	10
21. 数据质量分析	11
22. 局限性	11
23. 和 ReAct 的关系	12
24. 和本仓库代码的对应关系	12
25. 本仓库最小实验 1: 筛选候选工具调用	13
26. 本仓库最小实验 2: OpenAI function calling	13
27. 本仓库最小实验 3: MCP 协议栈	14
28. 本仓库最小实验 4: tool safety	14
29. Capstone 怎么读	15
30. 用 AI agent 学 Toolformer	15
31. 读完必须能回答	16
32. 学习节奏建议	16

guide Toolformer: Language Models Can Teach Themselves to Use Tools

原论文: Toolformer: Language Models Can Teach Themselves to Use Tools

本地原文 PDF: 已入库, 同目录文件 01_toolformer.pdf

作者: Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, Thomas Scialom

年份: 2023

本 guide 目标: 让你理解工具调用不是只靠 prompt 写几个函数说明。Toolformer 的关键问题是: 如何让语言模型用很少的人工示例, 自动给大规模文本标注 API calls, 再只保留真正能

帮助预测未来 token 的调用，最后学会何时、如何、用哪个工具。

0. 先给结论

Toolformer 的核心贡献可以压成一句话：

用少量 API 使用示例让模型在普通文本中自生成候选工具调用，执行这些调用，再用“工具结果是否降低未来 token loss”来筛选，最后把筛选后的 API call 插回文本中继续训练模型。

它解决的是一个很具体的问题：

- 大语言模型知道很多东西，但算术、日期、检索、翻译等能力不稳定。
- 外部工具能做这些事，但模型需要知道什么时候调用、调用什么、参数怎么写、结果怎么接着用。
- 人工标注大量工具调用轨迹太贵。
- Toolformer 试图让模型自己教自己。

一句话机制：

```
plain LM dataset
-> sample possible API calls
-> execute API calls
-> keep calls that reduce future-token loss
-> finetune LM on text with inserted API calls
-> during inference, LM emits API call, system executes tool, LM continues
```

这篇论文和 ReAct 的关系：

- ReAct 重点是在线交互: Thought -> Action -> Observation。
- Toolformer 重点是训练数据生成: 自监督学习何时插入 API call。
- ReAct 更像 agent loop。
- Toolformer 更像 tool-use pretraining / finetuning recipe。

今天的 function calling、MCP、computer use 和 tool safety，都可以把 Toolformer 当成早期思想来源之一：工具调用应该成为模型可学习的 token 行为，而不仅是外部硬编码流程。

1. 为什么需要工具

论文开头列出普通 LM 的几类限制：

- 不能访问最新事实，容易在近期事件上过时。
- 会 hallucinate factual statements。
- 低资源语言理解弱。
- 精确数学能力差。
- 对时间进展不敏感，例如日期、星期、当前年份。

这些问题继续 scaling 可以缓解一部分，但不彻底。一个计算器比大模型更擅长精确算术，一个搜索引擎比模型参数更适合查新事实，一个日历 API 比模型更可靠地知道今天日期。

问题不在于“工具有没有用”，而在于：

模型如何知道：

什么时候该调用工具？

调用哪个工具？

参数怎么写？

工具结果怎么放回上下文？

什么时候不该调用？

Toolformer 的目标是让这些行为通过自监督学习进入模型，而不是每个下游任务都人工写专门 prompt。

2. 论文的 desiderata

论文提出两条设计要求。

第一，工具使用应该 self-supervised。

原因不只是人工标注贵。更微妙的是：人类觉得有用的工具调用，不一定是模型觉得有用的工具调用。模型需要的是能降低自己预测未来 token 难度的信息。

第二，语言模型不应失去通用性。

工具使用不能变成只会解某个 benchmark 的特化系统。模型应该能在普通文本里自己决定是否调用工具，而不是用户必须提前告诉它“这个任务请调用计算器”。

所以 Toolformer 不对每个任务单独 finetune，而是在普通 LM 语料 CCNet 子集上插入工具调用，再继续用 language modeling objective 训练。

3. API call 的线性化

Toolformer 要把工具调用放进文本流里。论文把一个 API call 表示为：

```
c = (api_name, api_input)
```

不带结果的线性化：

```
<API> api_name(api_input) </API>
```

带结果的线性化：

```
<API> api_name(api_input) -> result </API>
```

论文中为了不修改 GPT-J 词表，实际用已有 token 序列表示这些特殊标记；为了讲解，我们用 <API>、</API>、->。

例子：

```
Out of 1400 participants, 400
(or <API> Calculator(400 / 1400) -> 0.29 </API> 29%)
passed the test.
```

这句话里，模型不是直接背出 29%，而是在文本中插入 calculator call，系统执行后把 0.29 放回文本，模型继续生成 29%。

4. 三步数据生成流程

Toolformer 的核心流程在 Figure 2:

1. Sample API Calls
2. Execute API Calls
3. Filter API Calls

输入是普通文本：

Pittsburgh is also known as the Steel City.

模型可能在“the Steel City”前采样多个候选 QA 调用：

```
QA("What other name is Pittsburgh known by?") -> Steel City
QA("Which country is Pittsburgh in?") -> United States
```

两个调用都语法上合理，但只有第一个对预测后面的“the Steel City”有直接帮助。Toolformer 执行两个调用，然后用 loss improvement 筛掉第二个。

保留下来的增强文本大概是：

```
Pittsburgh is also known as
<API> QA("What other name is Pittsburgh known by?") -> Steel City </API>
the Steel City.
```

这就是自标注的训练数据。

5. Step 1: 采样候选 API call

对每个 API，作者写一个 prompt，给少量示例，要求模型给普通文本添加 API calls。

例如 QA prompt 的意思是：

```
Your task is to add calls to a Question Answering API to a piece of text.
The questions should help you get information required to complete the text.
You can call the API by writing [QA(question)].
```

然后给少数示例，例如：

```
Input:
Joe Biden was born in Scranton, Pennsylvania.
```

```
Output:
Joe Biden was born in [QA("Where was Joe Biden born?")] Scranton,
[QA("In which state is Scranton?")] Pennsylvania.
```

采样位置的逻辑：

```
For each position i in text:
  compute probability that model starts an API call at i
  keep positions whose API-start probability > tau_s
  if too many positions, keep top k
```

默认 appendix 细节：

- $\tau_s = 0.05$ 。
- API-start probability 至少 5%。
- top k = 5 positions。
- 每个位置采样最多 $m = 5$ API calls。

对 calculator 和 machine translation，因为有用位置较少，作者放宽：

- $\tau_s = 0.0$ 。
- $k = 20$ 。
- $m = 10$ 。
- $\tau_f = 0.5$ 。

这体现一个工程事实：不同工具的数据稀疏性不同，采样阈值不能一刀切。

6. Step 2: 执行工具

候选 call 生成后，必须执行。执行方式取决于工具：

- QA tool: 调另一个问答模型。
- Calculator: 执行数学表达式。
- Wikipedia Search: BM25 检索 Wikipedia snippets。
- Machine Translation: 调翻译模型。
- Calendar: 返回当前日期。

论文要求 API 输入和输出都能表示为文本。这样工具调用才能插回 LM 训练序列中。

这个约束非常重要。Toolformer 不是直接教模型操作任意二进制接口，而是把工具世界压成：

text input -> text output

这也是后来 function calling 和 MCP schema 的精神来源之一：工具必须有可序列化输入、可解释输出、可审计边界。

7. Step 3: 用 loss improvement 筛选

这是论文最重要的数学。

设一个候选调用发生在位置 i ，工具调用是 c_i ，工具结果是 r_i 。我们比较三种情况下模型预测未来 tokens 的难度：

1. 什么工具都不给。
2. 只给 API call，不给 result。
3. 给 API call 和 result。

论文定义：

```
L_plus = loss when prefix includes API call and result
L_minus = min(
    loss with no API call,
    loss with API call but no result
)
```

如果 result 真有帮助， L_{plus} 应该比 L_{minus} 小。筛选规则：

```
keep candidate if:
    L_minus - L_plus >= tau_f
```

默认 $\tau_f = 1.0$ 。对 calculator 和 MT 这类样本少的工具，作者设置 $\tau_f = 0.5$ 。

为什么要和“只给 API call 不给 result”比较？因为模型看到 QA("Where was Pittsburgh known by?") 这个问题本身也可能提示未来 token。Toolformer 要筛的是 result 是否真的有用，而不是 API call 文本本身是否暗示答案。

8. 未来 token 权重

Toolformer 不把整个后文都同等看待。论文用一个随距离衰减的权重，让靠近 API call 的未来 token 更重要：

```
raw_weight(t) = max(0, 1 - 0.2 * t)
weight(t) = raw_weight(t) / sum(raw_weight)
```

直觉：

- 工具调用应该帮助附近文本。
- 如果一个工具结果只在很远之后才有用，很难判断它是否真的相关。
- 加权能让 API call 更靠近它帮助预测的信息。

这也解释了为什么 Toolformer 学到的是“在合适位置插入工具调用”，而不是文档开头乱塞一堆工具结果。

9. Finetuning 和推理

筛选后，作者把所有 API calls 合并到增强数据集 C_star:

```
C:      plain CCNet text
C_star: same text, with useful API calls inserted
```

然后用标准 language modeling objective finetune GPT-J。

训练细节:

- 基础模型: GPT-J 6.7B。
- 语料: CCNet 子集。
- 每个 API 最多使用 25k examples。
- max sequence length 1024。
- effective batch size 128。
- learning rate 1e-5。
- 前 10% linear warmup。
- DeepSpeed ZeRO-3。
- 8 张 NVIDIA A100 40GB。
- BF16。
- 最多 2k steps。
- 每 500 steps 在 1000 个 CCNet dev examples 上评估 PPL，选择最佳 checkpoint。

推理时:

```
decode normally
if model emits API response marker "->":
    pause decoding
    execute corresponding API call
    insert result and closing token
    continue decoding
```

论文还做了一个 decoding tweak: 不只在 <API> 是 top-1 token 时才开启调用，而是当 <API> 在 top-k tokens 内时也允许调用。实验里常用 k = 10。同时每个输入最多允许一次 API call，避免模型陷入不断调用 API 的 loop。

10. 五个工具

Toolformer 使用五类工具。

Question Answering.

使用 Atlas，finetuned on Natural Questions。生成训练数据时用 Atlas-large，推理时用 Atlas-xxl。

Calculator.

简单 Python script，只支持四则运算，结果保留两位小数。为了高效采样，只处理更可能有数学表达式的文本，例如 100 token window 内至少三个数字。

Wikipedia Search.

BM25 retriever, 检索 KILT Wikipedia dump。相比 QA tool, 它返回更丰富 snippets, 但模型必须自己抽取 relevant parts。

Machine Translation.

使用 600M NLLB, 把任意语言短语翻译成英文。用 fastText 检测源语言, 目标语言总是 English。

Calendar.

无输入, 返回当前日期。用于时间相关预测。

这些工具覆盖了不同短板:

- factual lookup。
- exact arithmetic。
- broad retrieval。
- low-resource multilingual understanding。
- temporal awareness。

11. 数据量和阈值

Table 2 展示不同 τ_f 下保留的 API-call examples 数量。

$\tau_f = 1.0$ 时:

- Question Answering: 18,526
- Wikipedia Search: 60,974
- Calculator: 994
- Calendar: 20,587
- Machine Translation: 1,034

$\tau_f = 0.5$ 时:

- Question Answering: 51,987
- Wikipedia Search: 207,241
- Calculator: 3,680
- Calendar: 61,811
- Machine Translation: 3,156

$\tau_f = 2.0$ 时:

- Question Answering: 5,135
- Wikipedia Search: 13,944
- Calculator: 138
- Calendar: 3,007
- Machine Translation: 229

你应该读出一个 tradeoff:

- 阈值低: 数据多, 但噪声多。
- 阈值高: 数据干净, 但样本少。
- Calculator 和 MT 特别稀疏, 需要额外 heuristic 和低阈值。

这也是现代 tool-use 数据构造的常见难题: 好的工具调用样本不是自然大量出现的, 要靠采样、过滤、执行、评分一起造。

12. 实验设置

论文比较的模型:

- GPT-J: 原始 GPT-J 6.7B。
- GPT-J + CC: 在 CCNet 子集 c 上继续训练, 但没有 API calls。
- Toolformer disabled: 在 C_star 上训练, 但推理时禁用 API calls。
- Toolformer: 在 C_star 上训练, 推理时允许 API calls。
- OPT 66B。
- GPT-3 175B davinci, 不是 instruction-tuned。

评估是 zero-shot prompting。没有给下游任务专门工具调用示例。这点很重要, 因为作者要证明模型学会了通用工具使用, 而不是每个任务靠 few-shot prompt 临时教。

13. LAMA: factual completion

LAMA 子集包括 SQuAD、Google-RE、T-REx。任务是补全一个缺失 fact。因为原始 LAMA 是 masked LM benchmark, 作者过滤成 left-to-right 可处理形式, 并用宽松指标: 正确词出现在前 5 个生成词里即可。

结果:

- GPT-J: SQuAD 17.8, Google-RE 4.9, T-REx 31.9。
- GPT-J + CC: 19.2, 5.6, 33.2。
- Toolformer disabled: 22.1, 6.3, 34.9。
- Toolformer: 33.8, 11.5, 53.5。
- OPT 66B: 21.6, 2.9, 30.1。
- GPT-3 175B: 26.8, 7.0, 39.8。

Toolformer 在三个子集上都明显超过同规模 GPT-J, 也超过更大的 OPT 和 GPT-3。论文说它主要使用 QA tool, 约 98.1% 的样本会问 QA。

这个结果的关键不是“GPT-J 变聪明了”, 而是模型学会在缺 facts 时向工具要 facts。

14. 数学任务

数学 benchmarks:

- ASDiv。
- SVAMP。
- MAWPS。

结果:

- GPT-J: 7.5, 5.2, 9.9。
- GPT-J + CC: 9.6, 5.0, 9.3。
- Toolformer disabled: 14.8, 6.3, 15.0。
- Toolformer: 40.4, 29.4, 44.0。
- OPT 66B: 6.0, 4.9, 7.9。
- GPT-3 175B: 14.0, 10.0, 19.8。

允许 Toolformer 调 calculator 后, 三个任务性能都大幅上升, 而且超过更大模型。论文报告数学任务中, 模型 97.9% 的例子会调用 calculator。

有趣的是 Toolformer disabled 也比 GPT-J 强。这说明只是在训练中见过 API call 和 result, 也可能让模型内部数学模式变好。但真正大提升来自推理时实际调用 calculator。

15. 问答任务

QA benchmarks:

- Web Questions。
- Natural Questions。
- TriviaQA。

为了避免 QA tool 直接作弊，作者禁用 question answering tool，因为底层 QA 系统本来就 finetuned on Natural Questions。Toolformer 主要使用 Wikipedia Search。

结果:

- GPT-J: WebQS 18.5, NQ 12.8, TriviaQA 43.9。
- GPT-J + CC: 18.4, 12.2, 45.6。
- Toolformer disabled: 18.9, 12.6, 46.7。
- Toolformer: 26.3, 17.7, 48.8。
- OPT 66B: 18.6, 11.4, 45.7。
- GPT-3 175B: 29.0, 22.6, 65.9。

Toolformer 超过同规模模型，但不如 GPT-3。论文解释: Wikipedia Search API 很简单，常返回不匹配结果；Toolformer 也不能交互式重写 query 或浏览多个结果。

这正是 Toolformer 的边界: 它学会单次工具调用，但还不是 ReAct 式多步搜索 agent。

16. 多语言问答

MLQA 设置:

- context paragraph 是英文。
- question 可能是 Spanish、German、Hindi、Vietnamese、Chinese、Arabic。
- 模型需要理解非英文问题，并用英文回答。

Toolformer 可以调用 machine translation tool，把问题翻成 English。

结果显示，使用 API calls 对各语言都有帮助，但 Toolformer 不总是超过原始 GPT-J。原因是继续在 CCNet 上训练可能对某些语言造成分布偏移，损害 GPT-J 原有多语言能力。

论文还指出 OPT 和 GPT-3 在这个设置下很弱，主要因为它们没有按要求用英文回答；当把 context 和 question 都给英文版本时，GPT-3 又变强。

这一节的意义是: 工具有帮助，但继续训练数据分布也可能伤害已有能力。

17. 时间任务

时间 benchmarks:

- TEMPLAMA。
- DATESET，作者构造的新数据集。

TEMPLAMA 是随年份变化的 Wikidata facts，例如某运动员在哪个队。DATESET 是日期推理模板，例如“某日期是几天前”、“某日期是星期几”。

结果:

- GPT-J: TEMPLAMA 13.7, DATESET 3.9。
- GPT-J + CC: 12.9, 2.9。

- Toolformer disabled: 12.7, 5.9。
- Toolformer: 16.3, 27.3。
- OPT 66B: 14.5, 1.3。
- GPT-3 175B: 15.5, 0.8。

DATESET 上 calendar tool 非常有效, Toolformer 使用 calendar 约 54.8%。

但 TEMPLAMA 的提升不是主要来自 calendar, 因为 calendar 只用了约 0.2%。很多 TEMPLAMA facts 涉及具体实体, 知道当前日期也不够, 还需要 QA 或 search。

这说明工具选择本身是难题。一个任务看起来是时间任务, 不代表只靠 calendar 就能解。

18. 语言建模能力是否退化

Toolformer 在 C_star 上训练, 里面插了 API calls。一个担忧是: 模型会不会失去普通 language modeling 能力?

作者在 WikiText 和 CCNet validation 上看 perplexity:

- GPT-J: WikiText 9.9, CCNet 10.6。
- GPT-J + CC: WikiText 10.3, CCNet 10.5。
- Toolformer disabled: WikiText 10.3, CCNet 10.5。

结论: 插入工具调用继续训练, 不会在禁用 API calls 时显著损害普通 LM perplexity。

这很重要, 因为 Toolformer 的目标不是把模型变成单任务工具机器人, 而是在保留通用性的同时增加工具使用能力。

19. Scaling: 多大模型才会用工具

作者把方法应用到 GPT-2 系列小模型:

- 124M。
- 355M。
- 775M。
- 1.6B。
- GPT-J 6.7B。

只用三种工具: QA、calculator、Wikipedia Search。

Figure 4 的结论:

- 最小模型几乎不能有效利用工具。
- 工具使用能力大约在 775M 参数附近开始出现。
- 模型越大, 不用工具时能力也增强。
- 但即使模型变大, 允许工具调用和禁用工具之间仍有明显差距。

这告诉你: 工具使用本身也是能力。不是给小模型一个工具接口, 它就会自动用好。

20. Decoding strategy

Toolformer 推理时有一个重要细节: 如果 <API> 不是 top-1 token, 普通 greedy decoding 可能永远不调用工具。所以作者允许当 <API> 位于 top-k tokens 中时也触发 API call。

Table 9:

T-REx:

- k = 0: overall 34.9, call rate 0.
- k = 1: overall 47.8, call rate 40.3%.
- k = 3: overall 52.9, call rate 82.8%.
- k = 10: overall 53.5, call rate 98.1%.

WebQS:

- k = 0: overall 18.9, call rate 0.
- k = 1: overall 19.3, call rate 8.5%.
- k = 3: overall 26.3, call rate 99.3%.
- k = 10: overall 26.3, call rate 100%.

有趣的是 k = 1 时, 模型有一定 calibration: 它选择不调用 API 的样本, 本来就比平均更容易。但 k 变大后, call rate 接近全量, calibration 会变弱。

这对现代 tool agents 也很重要: tool-call threshold 是行为开关。阈值太高, 模型不敢调用; 阈值太低, 模型乱调用。

21. 数据质量分析

Table 10 展示多个 API call examples, 并按 L_minus - L_plus 排序。

高分例子通常直观有用:

- WikiSearch 找到 Flodden war memorial。
- Calendar 返回日期帮助预测开放日。
- QA 查询 Nile 长度。
- Calculator 计算 735 / 499。
- MT 翻译非英语短语。

低分或负分例子通常没用:

- Calendar 返回日期但对后文没帮助。
- Calculator 结果和后文并不一致。
- QA 问题抽取错了。

论文也提到有噪声不一定全坏。训练数据中少量无用调用可能让模型学会不要盲从每个 tool result。

不过这也暴露风险: loss-based filtering 不等于语义正确。一个工具结果可能降低 perplexity, 但事实无关或误导。

22. 局限性

论文第 7 节很清楚地列出局限。

不能链式用工具。

Toolformer 的 API calls 是独立生成的。训练数据里没有“先用 calendar 得到日期, 再把日期送进 QA”这种链式样本。

不能交互式使用工具。

搜索工具可能返回很多结果, 但 Toolformer 不能浏览、改 query、继续查。这和 ReAct/WebGPT 不同。

对输入措辞敏感。

模型是否调用 API 往往受 wording 影响。这和 zero-shot/few-shot LM 本身对 prompt 敏感一致。

样本效率低。

处理超过百万文档，calculator 最后可能只得到几千个有用调用。

不考虑工具成本。

决定是否调用 API 时，没有把工具延迟、费用、失败率、安全风险纳入目标。

工具结果不保证安全。

论文主要关注预测性能，没有完整处理工具注入、权限、sandbox、审计和风险控制。

这些局限正好解释了本仓库为什么还要讲 MCP、sandbox、retry、tool injection 和 computer use。

23. 和 ReAct 的关系

Toolformer 和 ReAct 都是工具使用经典论文，但关注点不同。

Toolformer:

- 训练阶段核心。
- 自动生成工具调用数据。
- 用 loss improvement 筛选。
- 学会单次 API call。
- 适合把工具使用变成模型能力。

ReAct:

- 推理阶段核心。
- 在线 thought-action-observation loop。
- observation 改变下一步 action。
- 支持多步交互。
- 适合 agent 执行任务。

可以这样组合:

Toolformer teaches a model the grammar and usefulness of tool calls.

ReAct organizes tool calls into a multi-step task-solving loop.

MCP/function calling gives the runtime protocol to execute those calls safely.

24. 和本仓库代码的对应关系

本模块核心文件:

- learning/tool-use-mcp/src/toolformer_toy.py
- learning/tool-use-mcp/src/openai_tools.py
- learning/tool-use-mcp/src/mcp_protocol.py
- learning/tool-use-mcp/src/mcp_server.py
- learning/tool-use-mcp/src/mcp_client.py
- learning/tool-use-mcp/src/tool_retry.py
- learning/tool-use-mcp/src/tool_injection_demo.py
- learning/tool-use-mcp/src/sandbox_mock.py
- learning/tool-use-mcp/src/computer_use_mock.py
- learning/tool-use-mcp/src/capstone_mcp_stack.py

toolformer_toy.py 对应论文核心机制:

- APICandidate: 一个候选工具调用。
- linearize_call: 生成 `<API> name(arg) -> result </API>`。
- toolformer_score: 计算 $L_{\text{minus}} - L_{\text{plus}}$ 。
- keep_candidate: 根据 τ_f 保留或丢弃。
- interleave_call: 把保留的 API call 插回 token 序列。

这个 toy 文件不训练模型，但它让论文最核心的筛选逻辑能在本机跑。

25. 本仓库最小实验 1: 筛选候选工具调用

跑:

```
python learning\tool-use-mcp\src\toolformer_toy.py
```

里面有两个候选:

```
QA("What other name is Pittsburgh known by?") -> Steel City
```

```
QA("Which country is Pittsburgh in?") -> United States
```

第一个能显著降低预测“the Steel City”的 loss，所以 score 高于 τ_f 。

第二个虽然事实正确，但对当前位置后面的 token 没帮助，所以被过滤。

你要理解:

Toolformer 不是筛“工具返回是否真实”，而是筛“工具结果是否帮助模型预测未来文本”。

这是优点，也是局限。

26. 本仓库最小实验 2: OpenAI function calling

openai_tools.py 展示 function calling 的基本形状:

```
ToolSchema(
    name="add",
    description="add",
    input_schema={
        "type": "object",
        "properties": {
            "a": {"type": "number"},
            "b": {"type": "number"},
        },
    },
)
```

它可以转成 OpenAI-style tool schema，并解析:

```
{"id": "call_1", "function": {"name": "add", "arguments": "{\"a\":3,\"b\":4}"}}
```

这对应 Toolformer 的现代工程版本:

- Toolformer 用文本 `<API> ... </API>`。
- Function calling 用结构化 JSON schema。
- 二者都要求模型学会工具名、参数和结果接入。

27. 本仓库最小实验 3: MCP 协议栈

mcp_protocol.py 是 JSON-RPC 2.0 envelope:

```
make_request(id, method, params)
make_response(id, result)
make_error(id, code, message)
```

mcp_server.py 支持:

- initialize
- tools/list
- tools/call

mcp_client.py 做:

- initialize。
- list_tools。
- call_tool。

这说明从 Toolformer 到 MCP 的演化:

Toolformer:

model learns textual API call grammar.

MCP-style stack:

runtime exposes tools through discoverable protocol.
client lists tools, calls tools, handles errors.

Toolformer 关心“模型怎么学会调用”。MCP-style stack 关心“工具如何被发现、调用、返回、报错”。

28. 本仓库最小实验 4: tool safety

tool_injection_demo.py 演示 prompt injection / tool output injection 检测:

- 检测“ignore previous instructions”。
- 检测输出 secret/token/password 的请求。
- 删除 zero-width 控制字符。
- 删除 hidden HTML comment。
- 截断过长 tool output。

sandbox_mock.py 演示受限 Python 执行:

- AST 检查禁止 import。
- 禁止 eval、exec、open。
- 禁止 dunder attribute。
- 限制 builtins。

tool_retry.py 演示:

- transient failure retry。
- exponential backoff。
- permanent error 不重试。
- circuit breaker。

这些都不是 Toolformer 论文重点，但是真实工具系统必须有。模型学会调用工具以后，系统还要保证:

- 工具输出不能劫持模型。
- 工具调用不能越权。
- 工具失败要可恢复。
- 危险执行要 sandbox。
- 错误要结构化返回。

29. Capstone 怎么读

跑:

```
python learning\tool-use-mcp\src\capstone_mcp_stack.py
```

它会:

1. 建一个 in-memory MCP server。
2. 注册三个工具: calculator、search_kb、get_time。
3. client initialize。
4. client list_tools。
5. 调用三个合法工具。
6. 调用一个 bogus_tool, 确认错误被捕获。

这不是 Toolformer 训练流程, 而是工具运行时协议栈。你要把它和论文这样对应:

Toolformer paper:

model learns when to emit a tool call.

capstone_mcp_stack:

once a tool call exists, runtime discovers, executes, and returns result.

一个完整 agent 需要两边:

- 学会何时调用。
- 安全可靠地执行调用。

30. 用 AI agent 学 Toolformer

不要让 agent 只总结。这篇最值得被考的是筛选公式和实验边界。

推荐提示词:

我正在读 Toolformer。

请一次只问我一个问题, 并要求我把答案对应到本仓库代码。

问题必须覆盖:

1. Toolformer 为什么需要 self-supervised tool use。
2. API call 如何线性化。
3. 如何采样 API call positions。
4. L_plus、L_minus、tau_f 分别是什么。
5. 为什么只给 API call 不给 result 也要作为 baseline。
6. Toolformer 在 LAMA、数学、QA、DATESET 上各靠什么工具提升。
7. 为什么它不能链式或交互式用工具。
8. 它和 ReAct、MCP 的区别。

闭卷复述目标:

Toolformer 用少量 API 示例让 GPT-J 在普通 CCNet 文本中生成候选工具调用。

候选调用会被真实执行，得到文本结果。

论文用未来 token loss 做筛选：如果带工具结果的 loss 明显低于不调用或只给调用不给结果的 loss，就保留这

保留的调用被插回原文本，形成 C_star，再继续用语言建模目标训练。

推理时模型可以生成 API call，系统执行工具并把结果插回上下文。

实验显示 Toolformer 在 factual completion、数学、问答、日期任务上明显超过同规模 GPT-J，并在若干任务上

31. 读完必须能回答

你应该能闭卷回答：

1. Toolformer 和 ReAct 的核心差别是什么？
2. 为什么工具调用数据不能完全靠人标？
3. 一个 API call 在论文中如何线性化？
4. L_plus 是什么？
5. L_minus 为什么取两个 baseline 的 min？
6. tau_s 和 tau_f 分别控制什么？
7. 为什么 calculator 和 MT 要降低阈值/增加采样？
8. Toolformer 为什么在数学任务上提升大？
9. 为什么 QA 任务禁用了 QA tool？
10. DATESET 为什么适合 calendar tool？
11. 为什么 TEMPLAMA 不主要靠 calendar？
12. 为什么 Toolformer 不会明显损害 LM perplexity？
13. 为什么小模型不一定会用工具？
14. k 值增大为什么会提高 API call rate？
15. 本仓库哪个文件对应 L_minus - L_plus 筛选？
16. 本仓库哪个文件对应 runtime tool protocol？
17. 工具系统为什么需要 injection detection 和 sandbox？

32. 学习节奏建议

第一遍读机制：

- Figure 1。
- Figure 2。
- Section 2。
- 本 guide 第 3 到第 9 节。
- toolformer_toy.py。

目标是讲清楚从 plain text 到 C_star 的每一步。

第二遍读证据：

- Table 2 数据量。
- Table 3 LAMA。
- Table 4 数学。
- Table 5 QA。
- Table 7 时间任务。
- Table 8 perplexity。
- Figure 4 scaling。

目标是知道工具具体在哪些任务上帮了忙，哪里没帮够。

第三遍读工程:

- `openai_tools.py`.
- `mcp_protocol.py`.
- `mcp_server.py`.
- `mcp_client.py`.
- `tool_injection_demo.py`.
- `sandbox_mock.py`.
- `capstone_mcp_stack.py`.

目标是把 Toolformer 的“学会调用”接到真实工具系统的“安全执行”。

真正掌握的标志是: 你能看到一个 tool call, 不只问“它返回对不对”, 还会问“模型为什么在这里调用、结果是否降低任务不确定性、是否需要链式调用、输出是否可信、执行是否安全、失败是否可恢复”。